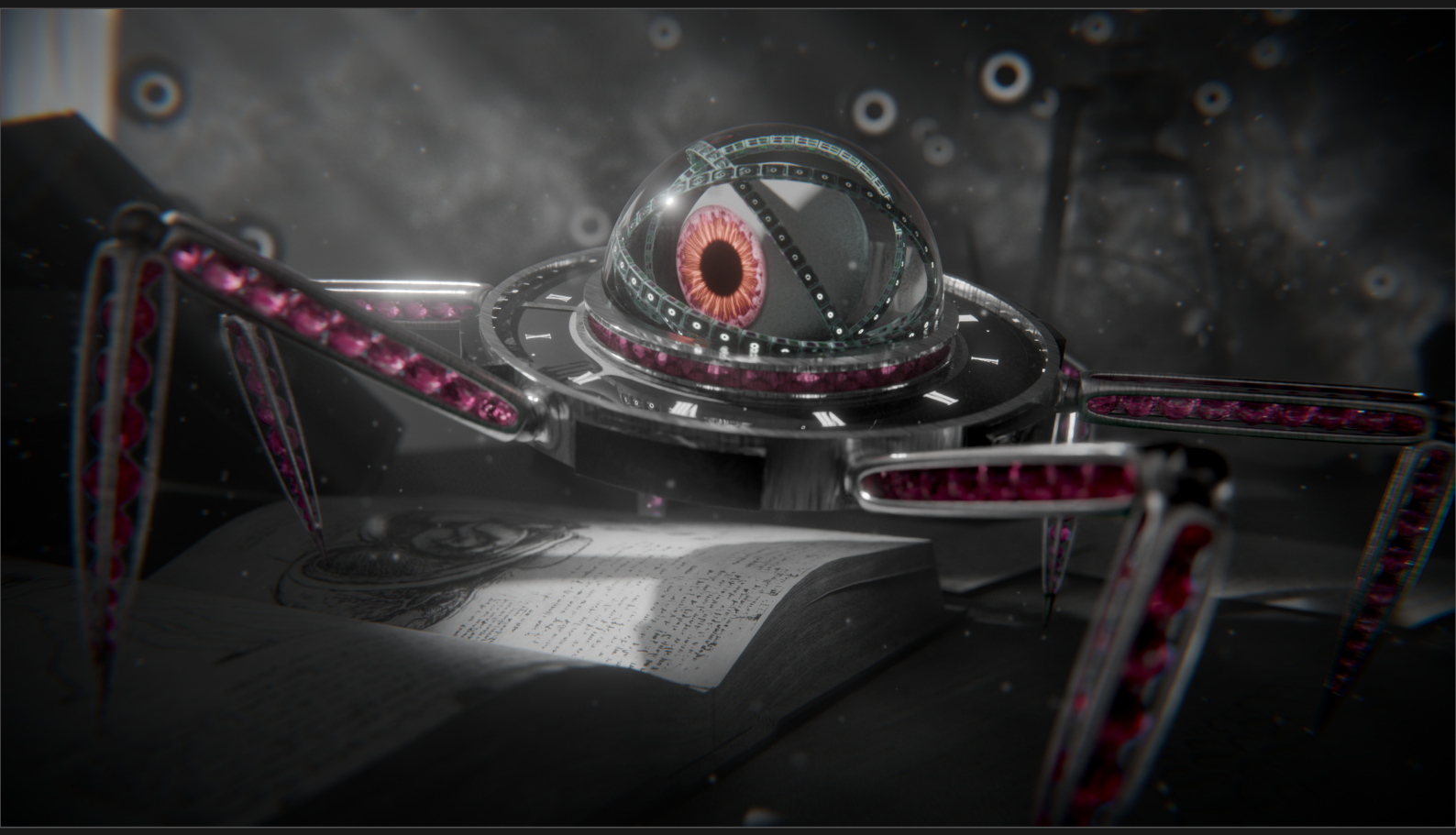
TECHNICAL PORTFOLIO

FABRIK

A procedural kinematic locomotion solver

Iterative distance constraints, indexed state and spatial transforms implemented as a visual programming project in Blender Geometry Nodes.



Rendered configuration: 6 legs, with 3 points per leg forming 2 segments.

n	N - 1	K
PROCEDURAL LEG COUNT	SEGMENTS PER CHAIN	ITERATION BUDGET

A parameterized model of reach, timing and three-dimensional geometry

01 / SYSTEM ARCHITECTURE

Topology, state and constraint flow

Coordinating many articulated legs manually led me to build a procedural locomotion system through visual programming in Blender Geometry Nodes. Its mechanics separate persistent foot state from the inverse-kinematics solve: a phase schedule updates terrain-aware foot targets, their geometry defines a shared root center, and FABRIK resolves each chain. Leg count n and joint count N parameterize the topology. The shown preset uses 10 legs and 4 joints per chain; the solver derives segment lengths and loop bounds from N.

1.1 The dependency order

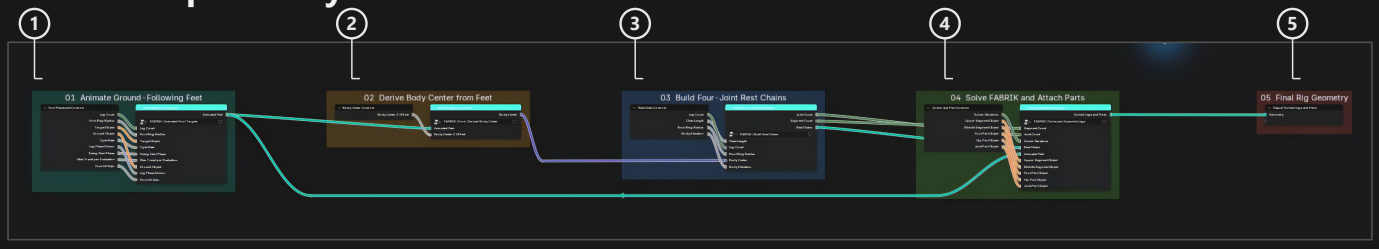


Figure 1. The top-level FABRIK tree. Numbered stages follow the data dependencies below.

1	**Foot state**	Maintain n indexed positions, gate their target-directed updates and project onto terrain.
2	**Root geometry**	Reduce the animated foot positions to a centroid, then translate the root ring.
3	**Chain topology**	Resample each rest chain to N joints; store d = L / (N - 1) and initial positions.
4	**Constraint solve**	Process each spline with K pairs of sequential backward and forward distance corrections.
5	**Spatial output**	Convert solved points and curve normals into mesh positions, orientations and scale.

1.2 Rest data remains separate from solved positions

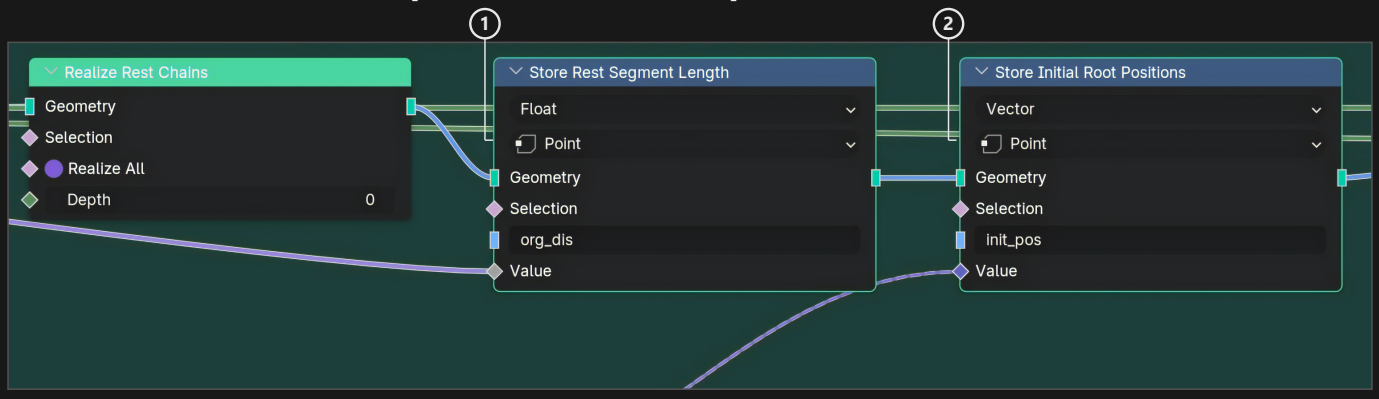


Figure 2. For N >= 2, org_dis stores d = L / (N - 1); init_pos stores the initial joint positions used to restore the root. This preset uses L = 1.35 and N = 4, giving d = 0.45 units.

02 / INVERSE KINEMATICS

Solving a chain with FABRIK

FABRIK, or Forward And Backward Reaching Inverse Kinematics, updates joint positions directly [1]. A chain has N joints and N - 1 distance constraints. The end effector should reach its target while joint 0 remains attached to the root. Alternating sweeps reconcile those endpoint conditions with the stored segment lengths. N determines both the resampling resolution and the number of corrections per sweep.

2.1 One distance constraint

$$r = d / ||p - q||$$

$$p_{new} = r\,p + (1 - r)\,q$$

p: joint being updated
q: neighboring joint
d: stored rest length

This places p exactly d units from q when their separation is nonzero. The weights need not stay between 0 and 1. Sweep order matters: each correction uses an already-updated neighbor, so a parallel field update would produce a different result.

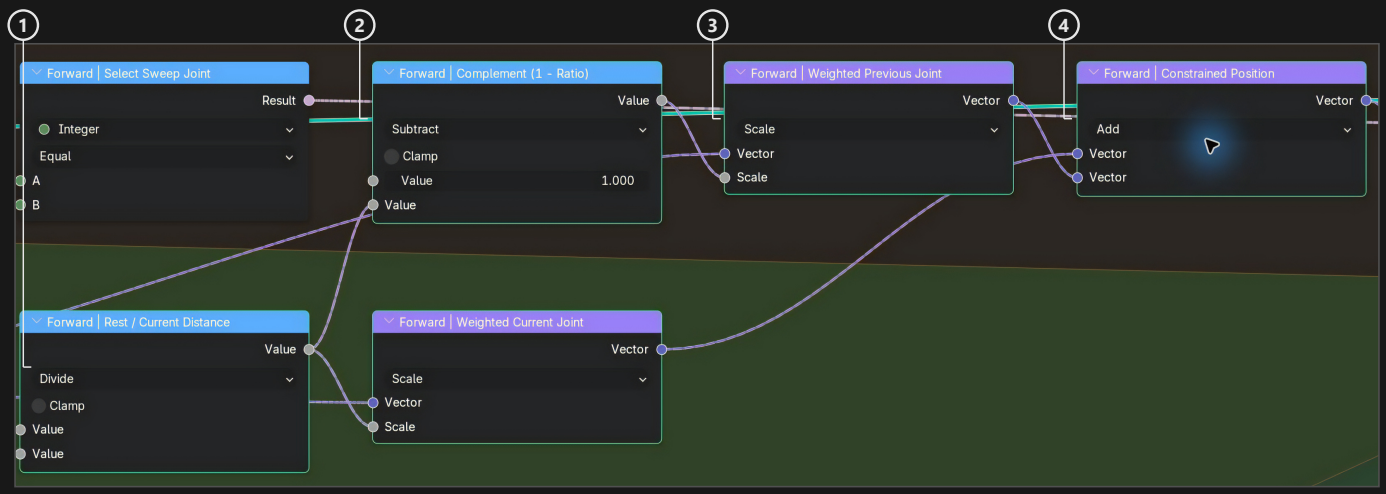


Figure 3. The forward-sweep kernel: (1) rest/current distance, (2) its complement, (3) weighted neighbor, and (4) sum with the weighted current joint.

2.2 A sequential pair of sweeps

1

Backward sweep

Pin joint N - 1 to the foot target. Visit i = N - 2 down to 0, constraining each p_i relative to p_(i+1).

2

Root boundary

Restore p_0 from init_pos. The backward sweep can move the root, so its boundary condition must be reimposed.

3

Forward sweep

Visit i = 1 through N - 1, constraining p_i relative to p_(i-1). The tip may move away from its target again.

A sweep pair performs 2(N - 1) distance corrections. Across n legs and K iterations, the count is 2nK(N - 1): 900 in the shown preset. There is no error-based early stopping.

03 / GAIT SCHEDULING

Persistent state and phase scheduling

The foot subsystem evolves through a discrete state update. A simulation zone retains one position per leg; a periodic phase determines when each indexed state may move toward its target. Keeping point indices consistent across the foot ring and solved splines is essential: leg i must sample the target belonging to foot i.

3.1 Phase and swing window

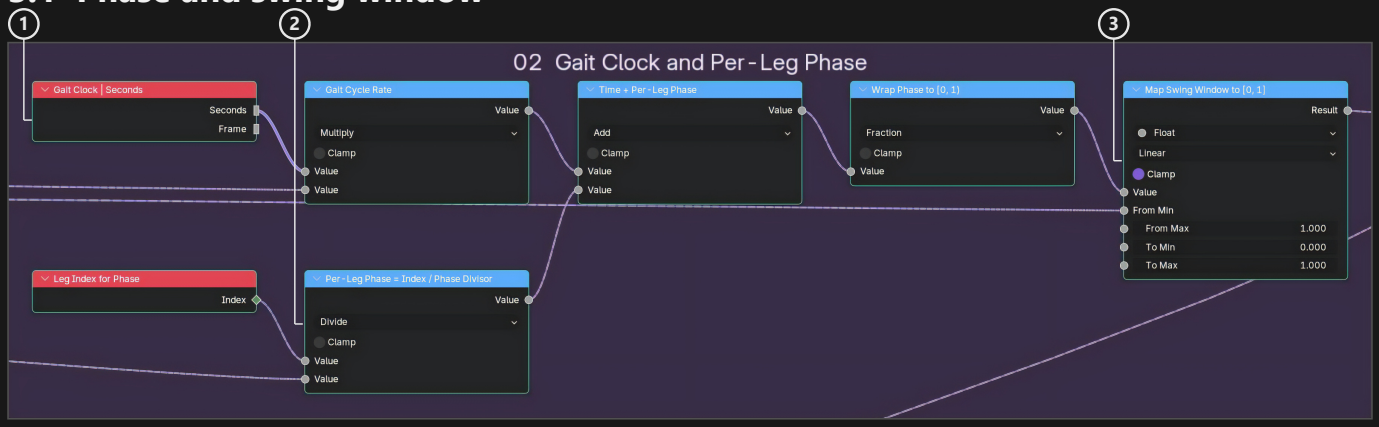


Figure 4. (1) Scene seconds supply the clock. (2) Leg index / phase divisor staggers the legs. (3) A clamped Map Range converts the swing window to a local phase.

$$\text{phase}_i = \text{fract}(t\,f + i / D)$$

$$u_i = \text{clamp}((\text{phase}_i - s) / (1 - s), 0, 1)$$

t is scene time in seconds. Defaults: f = 1 cycle/s, D = 4, s = 0.5. D is independent of n.

3.2 Lift follows the amount of movement



lift = C(u) x gain x |offset|

The profile C(u) shapes a vertical excursion. Its amplitude scales with the post-query displacement, so lift and travel are coupled. Computing the raised output after the simulation zone separates the swing trajectory from the ground-projected foot history.

Figure 5. The actual Blender Float Curve, sampled at 21 points. The peak is 1 at u = 0.5; Foot Lift Gain is a multiplier, not a height setting.

The near-endpoint comparison gate excludes the ends of swing. Outside swing, horizontal target-driven travel is disabled, while the terrain projection may still change height.

04 / TERRAIN QUERIES

A bounded update with terrain projection

Each foot update combines a target-directed displacement with a terrain query. The displacement is bounded before projection; the ray then replaces the proposed position with a surface hit when one exists. Separating proposal, projection and transient lift makes the state transition explicit. A ray miss has a defined fallback, while a hit is a positional constraint rather than evidence of a physically valid contact.

4.1 Propose, query, choose

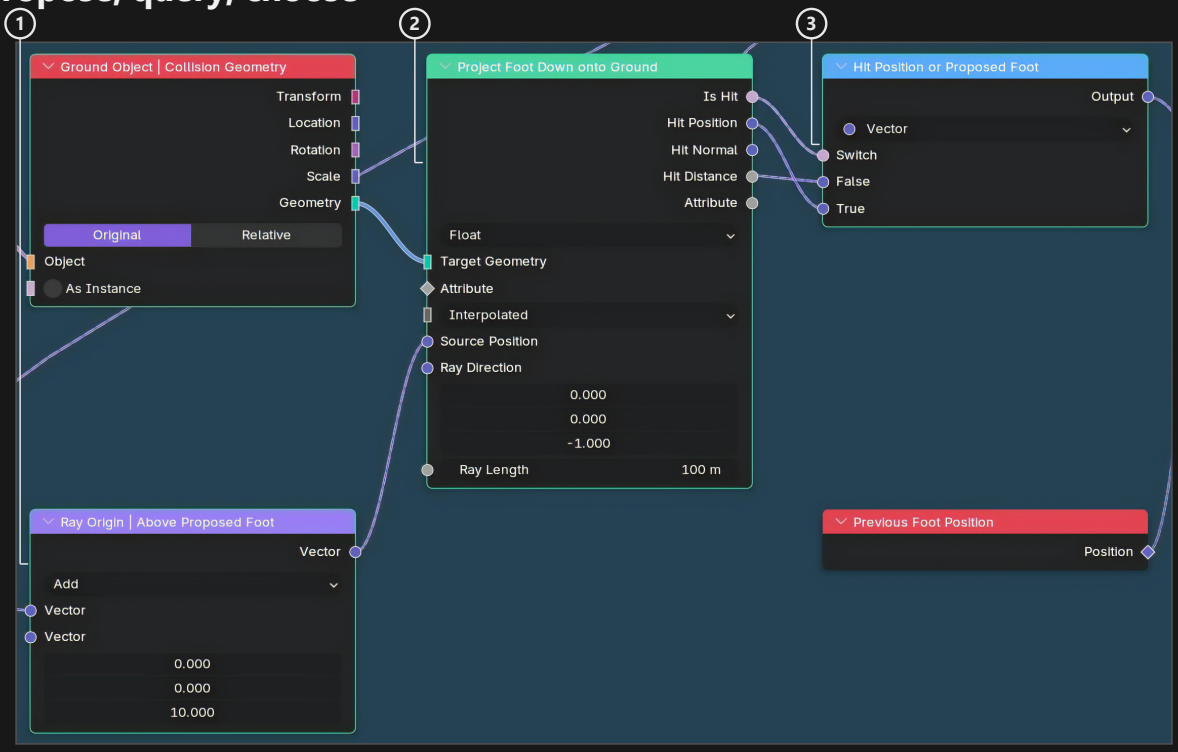


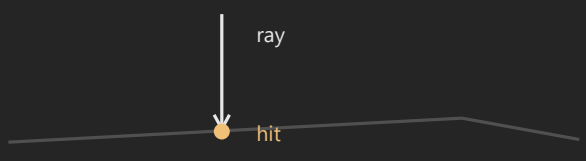
Figure 6. (1) Ray origin = proposal + (0, 0, 10). (2) Downward Raycast uses a 100-unit range. (3) A switch returns Hit Position on success, or the proposed position on a miss.

Proposal q

For desired foot a and stored foot p:
q = p + g normalize(a - p)
x min(||a - p||, v_max).
g is the swing gate; v_max is the per-evaluation travel cap.

Next state

p_next = Hit Position on a hit, otherwise q. Store offset = p_next - p and apply it to the persistent state. The terrain projection can change the displacement magnitude after the proposal was capped.



Space matters. The target ring is relative to IK; the ground geometry uses original space. IK and Ground currently have identity transforms. Independent object transforms would require a space-consistency check.

Figure 7. Schematic of the vertical ground query. It does not check the entire swing arc for obstacles.

05 / SPATIAL CONSTRUCTION

Spatial reference frames and geometry

5.1 A foot-derived center

The root center is a geometric reduction over the animated foot positions: c = (1/n) sum_i f_i. The node implementation duplicates the foot vertices, merges the selected duplicates into one center, then removes the originals. A Z offset positions the root ring above c. Because f_i includes swing lift, this estimate is not a center of mass or a support-polygon balance calculation.

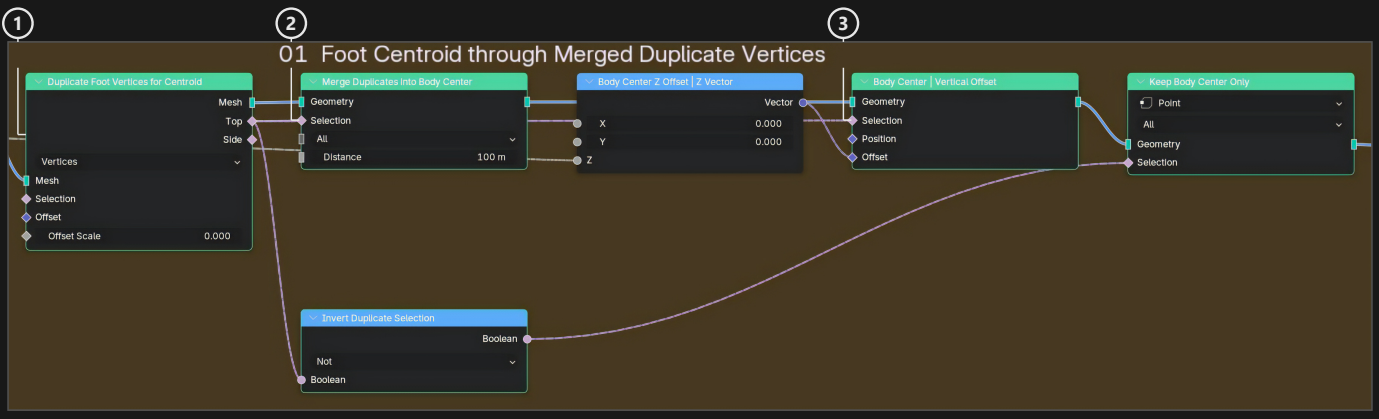


Figure 8. (1) Duplicate vertices, (2) merge them into one center, and (3) offset the center before deleting the original vertices. The existing merge threshold is 100 units.

root center = mean(animated feet) + (0, 0, 0.3)

The 0.3-unit net offset is Body Center Z Offset (-0.1) plus Body Elevation (0.4).

5.2 Recovering an orientation from geometric fields

A segment direction fixes an axis but leaves rotation about that axis undetermined. The part group uses p_i - p_(i+1) to align local Y, then uses the curve normal to establish local X as a roll reference. This converts point geometry into an orientation in three dimensions. org_dis sets uniform scale; the source meshes must have compatible local axes and dimensions.

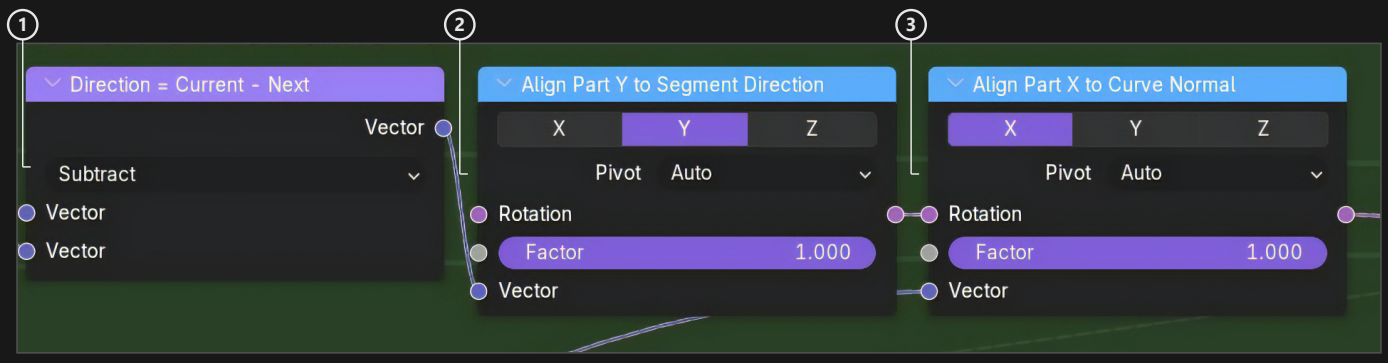


Figure 9. (1) Segment direction, (2) Y alignment and (3) normal-based X alignment. Mesh objects select joint indices, so changing the procedural joint count requires remapping those assignments.

06 / PARAMETERIZATION

Parameters and their mathematical roles

The visual programming graph exposes 19 Geometry Nodes modifier controls. Below are the current preset values, not topology limits. Joint count N is an internal procedural input; distances are in Blender units.

CONTROL	VALUE	ROLE
Leg Count	10	n: radial legs and indexed foot states.
Chain Length	1.35	Rest length L; d = L / (N - 1).
Root Ring Radius	0.44	Radius of the attachment ring.
Foot Ring Radius	1.22	Radius of initial and desired foot rings.
Body Elevation	0.4	Root height above the derived center.
Body Center Z Offset	-0.1	Vertical adjustment to the foot centroid.
Target Object	Target	Transform defining the desired foot ring.
Ground Object	Ground	Mesh used by the downward ground query.
Cycle Rate	1.0	Gait cycles per second.
Leg Phase Divisor	4.0	Phase offset = leg index / this value.
Swing Start Phase	0.5	Swing starts here and ends at phase 1.
Max Travel per Evaluation	0.1	Distance cap for each swing update.
Foot Lift Gain	7.3	Multiplies profile and actual step distance.
Solver Iterations	15	K: complete backward/forward pairs.
Upper Segment Object	Cylinder.014	Upper part at joint 0.
Middle Segment Object	Cylinder.014	Middle part at joint 1.
Foot Part Object	Cube.002	Foot part at joint 2.
Hip Part Object	Sphere.004	Hip part at joint 0.
Joint Part Object	Cylinder.012	Joint parts at joints 1 and 2.

Replay after structural changes

Changing Leg Count or Foot Ring Radius changes the simulation setup. Replay from frame 1 so stored positions correspond to the new configuration.

Procedural chain resolution

Joint count is fully procedural. Changing N requires remapping mesh objects to joint indices: assignments select by index, rather than by mesh-part role.

07 / SCOPE AND REFLECTION

What this system does and does not do

Implemented

Parameterized leg and chain topology; per-leg persistent state; periodic phase offsets; terrain projection; iterative segment-length constraints; and spatial orientation recovered from geometric fields.

Outside the current scope

No simulated mass, torque or ground-reaction forces. No dynamic balance controller, actuator model, joint-angle limits, self-collision or learned policy. This is a kinematic model of locomotion.

7.1 Current limitations

Heading The body orientation does not turn to follow travel. Moving the target behind the rig produces backward movement at the existing heading.

Update rate The travel cap is per evaluation and is not multiplied by delta time. Changing evaluation rate can change travel speed. Cycle timing itself uses seconds.

Reachability A target farther than total chain length L from the root is unreachable. The tree has no explicit unreachable-target branch. Fixed K also leaves residual tip error possible for reachable targets.

7.2 Connection to legged robotics

Legged locomotion couples foothold selection, base motion and joint reachability. This project addresses a geometric subset of that problem: indexed foot targets, phase coordination and chains that satisfy segment-length constraints. It does not optimize force or stability. In contrast, Grandia et al. combine terrain perception with nonlinear model predictive control for robot locomotion [2]. The connection is the continual replanning of spatial targets; deploying a robot would additionally require sensing, dynamics, contact feasibility and actuator constraints.

7.3 What I would build next

I would measure tip error e = ||p_(N-1) - target|| across chain resolutions and iteration budgets, then add tolerance-based stopping and reachability handling. I would also make travel time-step consistent, introduce joint limits and add heading control. These steps extend the current geometric model toward a controller with explicit accuracy and feasibility criteria.

REFERENCES

[1] Aristidou, A. and Lasenby, J. (2011). FABRIK: A fast, iterative solver for the Inverse Kinematics problem. Graphical Models, 73(5), 243-260.

[2] Grandia, R. et al. (2022). Perceptive Locomotion through Nonlinear Model Predictive Control. arXiv:2208.08373. References are clickable.